FC Fluxo de Caixa

DOCUMENTO 15

Blueprint técnico

Contrato compartilhado: nomes, portas, evento, recursos AWS e metas de SLO

PROJETO

Fluxo de Caixa — Desafio de Arquiteto de Soluções

AUTOR

Bruno Aquino

DATA

11/11/2026

REPOSITÓRIO

github.com/brunoaquino/fluxo-caixa

Java 25 · Spring Boot 4 · AWS ECS Fargate · Terraform · C4 · DDD · ADRs

Sumário

1. Visão geral 3
2. Stack 3
3. Layout do repositório 4
4. Bounded contexts 4
5. Modelo de domínio 4
6. Contrato do evento (Published Language) 5
7. APIs REST 5
8. Arquitetura interna dos serviços (Hexagonal / Ports & Adapters) 7
9. Recursos de mensageria 7
10. Variáveis de ambiente (contrato serviço ↔ infra) 7
11. Ambiente local (docker-compose em infra/local) 8
12. AWS (alvo) 8
13. Metas de NFR (SLOs) 9
14. Borda: CloudFront + WAF + VPC Origin (Revisão 2) 9
15. Ambientes e dimensionamento (us-east-1) 10
16. Rastreabilidade ponta a ponta (traces nos logs) 11
17. Rotação de segredos 11
18. Resiliência em código (Resilience4j) 12
19. Desempenho de banco e consultas 12
20. API Gateway (Revisão 3) 13

FONTE: DOCUMENTOS-BRUTOS/BLUEPRINT-TECNICO.MD

Blueprint Técnico de Referência — Fluxo de Caixa

> Fonte única de verdade para nomes, contratos e convenções compartilhadas entre serviços, infraestrutura e documentação. Qualquer divergência deve ser resolvida a favor deste documento.

1. Visão geral

Solução para um comerciante controlar o fluxo de caixa diário (lançamentos de débito e crédito) e consultar o saldo diário consolidado.

- **Estilo**: microsserviços por *bounded context*, CQRS entre escrita (Lançamentos) e leitura (Consolidado), integração assíncrona orientada a eventos.
- **Requisito crítico (RNF)**: o serviço de Lançamentos **não pode ficar indisponível** se o Consolidado cair. Em pico, o Consolidado recebe **50 req/s** com no máximo **5% de perda**.
- **Como o RNF é atendido**: Lançamentos não chama o Consolidado de forma síncrona. Ele grava o lançamento e o evento na mesma transação (Transactional Outbox) e um *relay* publica no SNS. O Consolidado consome via SQS (com DLQ), é idempotente e serve leituras a partir de uma projeção materializada + cache Redis. Se o Consolidado cair, as mensagens ficam retidas no SQS (retenção de 14 dias) e são processadas quando ele voltar.

2. Stack

Item	Escolha
Linguagem	Java 25
Framework	Spring Boot **4.0.9** (última 4.0.x) + **Resilience4j 2.4.0** (`resilience4j-spring-boot4` , compilado contra Boot 4.0.0 / `resilience4j-spring6`). Boot 4.1.x fica bloqueado até o Resilience4j publicar suporte — ver seção 18
Build	Maven com wrapper (`mvnw`), um projeto Maven independente por serviço
Banco	PostgreSQL 17 (Amazon RDS em produção), um banco por serviço, migrações Flyway
Mensageria	Amazon SNS (tópico) + Amazon SQS (fila + DLQ). Localmente: LocalStack. Cliente: AWS SDK for Java v2 (sem Spring Cloud AWS)
Cache	Redis 7 / Valkey (Amazon ElastiCache em produção), apenas no Consolidado
Segurança	OAuth2/OIDC, JWT validado pelos serviços (Spring Security Resource Server). AWS: Amazon Cognito. Local: Keycloak
Observabilidade	Micrometer + OpenTelemetry (OTLP), logs JSON estruturados, Actuator. AWS: ADOT/CloudWatch/X-Ray. Local: `grafana/otel-lgtm`

Item	Escolha
Execução AWS	ECS Fargate atrás de **CloudFront (com AWS WAF) → VPC Origin → ALB interno**, região `us-east-1` **(N. Virginia)**
IaC	Terraform (provider AWS 6.x), modular, ambientes `dev`, `hml`, `prod`, state remoto em S3
CI/CD	GitHub Actions (build once, promote dev → hml → prod)

3. Layout do repositório

```
fluxo-caixa/
├── README.md
├── Makefile
├── documentos-brutos/            # documentação bruta (fontes Markdown)
│   ├── blueprint-tecnico.md      # este arquivo
│   ├── 04-adr/                   # Architecture Decision Records (MADR)
│   └── ...
├── services/
│   ├── lancamentos-service/      # projeto Maven independente
│   └── consolidado-service/      # projeto Maven independente
├── infra/
│   ├── terraform/                # módulos + ambientes (dev, prod)
│   └── local/                    # docker-compose, init LocalStack, realm Keycloak
├── tests/
│   ├── load/                     # scripts k6
│   └── e2e/                      # smoke tests ponta a ponta (scripts)
└── .github/workflows/
```

4. Bounded contexts

Contexto	Tipo	Serviço	Responsabilidade
Lançamentos (Gestão de Caixa)	Core	`lancamentos-service`	Registrar créditos e débitos, estornos, consultar lançamentos. Fonte da verdade.
Consolidação Diária	Core/Supporting	`consolidado-service`	Manter a projeção do saldo diário por comerciante e servir relatórios.
Identidade e Acesso	Generic	Cognito / Keycloak	Autenticação, emissão de tokens, escopos.

Relação: Lançamentos é *upstream* (Published Language: eventos JSON versionados); Consolidado é *downstream* conformista com uma camada anticorrupção no adaptador de mensageria.

5. Modelo de domínio

Lançamentos

- **Agregado** `Lancamento` (imutável após criado):
  - `id` (UUID v7 ou v4), `comercianteId` (UUID), `tipo` (`CREDITO` | `DEBITO`), `valor` (Value Object `Dinheiro`: `BigDecimal` escala 2, `> 0`, moeda `BRL`), `dataLancamento` (`LocalDate`, data de com-

petência), `descricao` (1..255), `categoria` (opcional, até 60), `lancamentoEstornadoId` (UUID, nulo se não for estorno), `criadoEm` (`Instant`).

- **Invariantes**: valor positivo com no máximo 2 casas; descrição obrigatória; um lançamento só pode ser estornado uma vez; estorno de um estorno é proibido; estorno gera um novo lançamento de tipo oposto com o mesmo valor e mesma `dataLancamento` do original (o saldo do dia original é corrigido).
- Não existe update/delete de lançamento — correção é feita via estorno (trilha de auditoria).

Consolidado

- **Agregado** `SaldoDiario` chave (`comercianteId` , `data`): `totalCreditos` , `totalDebitos` , `saldo` (= créditos – débitos), `quantidadeLancamentos` , `versao` (optimistic locking), `atualizadoEm` .
- **Saldo acumulado**: calculado na consulta como soma dos `saldo` de todos os dias `<= data` do comerciante.
- Aplicação de evento é comutativa (soma), portanto a ordem de chegada não importa; a idempotência é garantida pela tabela `eventos_processados(event_id PK)` gravada na mesma transação da atualização.

6. Contrato do evento (Published Language)

Publicado no tópico SNS, entregue no SQS com **raw message delivery = true** (o corpo da mensagem SQS é exatamente o JSON abaixo). Atributo de mensagem SNS/SQS `eventType` = `LancamentoRegistrado` .

```
{
  "eventId": "0b8f6c1e-6d1a-4b8e-9a43-0f1b7e2f9c11",
  "eventType": "LancamentoRegistrado",
  "eventVersion": 1,
  "occurredAt": "2026-11-10T18:00:00.123Z",
  "lancamentoId": "5f0e1c7a-1d2b-4c3d-8e9f-0a1b2c3d4e5f",
  "comercianteId": "11111111-1111-1111-1111-111111111111",
  "tipo": "CREDITO",
  "valor": "150.00",
  "moeda": "BRL",
  "dataLancamento": "2026-11-10",
  "lancamentoEstornadoId": null
}
```

- `valor` é **string decimal** com 2 casas (evita perda de precisão).
- Um estorno é publicado como `LancamentoRegistrado` com tipo oposto e `lancamentoEstornadoId` pre-enchido — o Consolidado não precisa de lógica especial.
- Entrega *at-least-once*; consumidor deve ser idempotente por `eventId` .
- Evoluções: campos novos são aditivos; quebra de contrato exige `eventVersion` 2 e período de convivência.

7. APIs REST

Base path `/api/v1` . Erros em **RFC 9457 Problem Details** (`application/problem+json`). OpenAPI via springdoc (`/v3/api-docs` , `/swagger-ui.html`). O `comercianteId` **nunca** vem do corpo/URL: é extraído do JWT (claim configurável `app.security.comerciante-claim` , padrão `comerciante_id`).

lancamentos-service (porta 8081)

Método	Caminho	Escopo	Descrição
POST	`/api/v1/lancamentos`	`lancamentos:write`	Registra lançamento. Header opcional `Idempotency-Key` (UUID): a mesma chave e o mesmo corpo devolvem o mesmo recurso (201 na primeira, 200 na repetição); a mesma chave com corpo diferente responde **409**.
GET	`/api/v1/lancamentos/{id}`	`lancamentos:read`	Detalhe
GET	`/api/v1/lancamentos?data=YYYY-MM-DD&page=0&size=50`	`lancamentos:read`	Lista paginada por data
POST	`/api/v1/lancamentos/{id}/estornos`	`lancamentos:write`	Estorna lançamento (body opcional `{ "motivo": "..." }`). O header `Idempotency-Key` vale também aqui: a impressão digital é o id estornado mais o motivo.

Request POST: `{ "tipo": "CREDITO", "valor": 150.00, "dataLancamento": "2026-11-10", "descricao": "Venda balcão", "categoria": "VENDAS" }`

- `valor` na requisição é número JSON (desserializado para `BigDecimal`); nas respostas e no evento é string decimal com 2 casas.
- Idempotência: primeira criação retorna **201 Created**; repetição com a mesma `Idempotency-Key` e a mesma impressão digital retorna **200 OK** com o mesmo recurso. A coluna `idempotencia.impressao_digital` guarda o SHA-256 do corpo canônico. Corpo diferente com a mesma chave responde **409**.
- Lista paginada: `{ "conteudo": [...], "pagina": 0, "tamanho": 50, "totalElementos": 1, "totalPaginas": 1 }`.
- Estorno: a `descricao` do lançamento de estorno é o `motivo` informado ou, se ausente, `Estorno do lançamento {id}`.

consolidado-service (porta 8082)

Método	Caminho	Escopo	Descrição
GET	`/api/v1/consolidados/diario/{data}`	`consolidado:read`	Saldo do dia: `{ data, totalCreditos, totalDebitos, saldo, saldoAcumulado, quantidadeLancamentos, atualizadoEm }`. Dia sem movimento retorna 200 com zeros.
GET	`/api/v1/consolidados/diario?inicio=YYYY-MM-DD&fim=YYYY-MM-DD`	`consolidado:read`	Relatório do período (máx. 366 dias): `{ inicio, fim, dias: [ <mesmo formato do saldo do dia> ] }`

Escopos: autoridades `SCOPE_<prefixo><escopo>`. Propriedade `app.security.scope-prefix` (padrão vazio; na AWS/Cognito = `fluxo-caixa/`).

8. Arquitetura interna dos serviços (Hexagonal / Ports & Adapters)

Pacotes base: `br.com.fluxocaixa.lancamentos` e `br.com.fluxocaixa.consolidado`.

```
domain/          # entidades, value objects, eventos de domínio, exceções — sem Spring
application/     # casos de uso (ports in), ports out (interfaces)
adapters/
  in/web/        # controllers REST, DTOs, exception handler
  in/messaging/  # (consolidado) listener SQS
  out/persistence/  # JPA/JDBC
  out/messaging/    # (lancamentos) publicador SNS + outbox relay
  out/cache/        # (consolidado) Redis
config/
```

Regras verificadas por ArchUnit: `domain` não depende de Spring nem de `adapters`; `application` não depende de `adapters`.

9. Recursos de mensageria

No LocalStack os nomes são fixos. Na AWS o Terraform prefixa o ambiente (`fluxo-caixa-dev-` / `fluxo-caixa-prod-`). Os serviços não leem esses nomes: publicam em `LANCAMENTOS_TOPIC_ARN` e consomem `CONSOLIDADO_QUEUE_URL`.

Recurso	Nome no local
Tópico SNS	`fluxo-caixa-lancamentos-events`
Fila SQS	`consolidado-lancamentos-queue` (visibility 60s, retenção 14d, long polling 20s)
DLQ	`consolidado-lancamentos-dlq` (maxReceiveCount = 5)

Outbox (lancamentos): tabela `outbox_eventos(id UUID PK, aggregate_id, event_type, payload JSONB, criado_em, publicado_em NULL, tentativas INT, proxima_tentativa_em)`. O relay, a cada 500 ms, lê um lote de 100 com `SELECT ... FOR UPDATE SKIP LOCKED` onde `publicado_em` é nulo e `proxima_tentativa_em` já passou. Falha de publish grava a próxima tentativa com backoff exponencial até 60 s.

10. Variáveis de ambiente (contrato serviço ↔ infra)

Comuns: | Variável | Exemplo local | |---|---| | `SERVER_PORT` | 8081 / 8082 | | `DB_URL` | `jdbc:postgresql://localhost:5432/lancamentos` ou `/consolidado` | | `DB_USERNAME` / `DB_PASSWORD` | `lancamentos` / `lancamentos`, `consolidado` / `consolidado` | | `AWS_REGION` | `us-east-1` | | `AWS_ENDPOINT_URL` | `http://localhost:4566` no local. Na AWS a variável não é definida. Ausência ou string em branco faz o SDK usar o endpoint regional padrão; não se passa string vazia para `endpointOverride` | | `JWT_ISSUER_URI` | `http://localhost:8080/realms/fluxo-caixa` | | `APP_SECURITY_SCOPE_PREFIX` | vazio local; `fluxo-caixa/` na AWS | | `OTEL_EXPORTER_OTLP_ENDPOINT` | `http://localhost:4318` | | `SPRING_PROFILES_ACTIVE` | `local` / `aws` |

Específicas: | Serviço | Variável | Exemplo local | |---|---|---| | lancamentos | `LANCAMENTOS_TOPIC_ARN` | `arn:aws:sns:us-east-1:000000000000:fluxo-caixa-lancamentos-events` | | consolidado | `CONSOLIDADO_QUEUE_URL` | `http://localhost:4566/000000000000/consolidado-lancamentos-queue` | | consolidado | `REDIS_HOST` / `REDIS_PORT` / `REDIS_SSL` | `localhost` / `6379` / `false` |

Health checks: `/actuator/health/liveness` e `/actuator/health/readiness` (ALB usa readiness). Métricas: `/actuator/prometheus`.

11. Ambiente local (docker-compose em `infra/local`)

Container	Porta	Observação
postgres	5432	cria bancos/usuários `lancamentos` e `consolidado`
redis	6379	
localstack	4566	cria tópico, fila, DLQ e subscription com raw delivery
keycloak	8080	realm `fluxo-caixa`, client `fluxo-caixa-app` (public, direct access grants), client scopes `lancamentos:write`, `lancamentos:read`, `consolidado:read` como default; usuário `comerciante1` / `comerciante1` com atributo `comerciante_id=11111111-1111-1111-1111-111111111111` mapeado como claim no access token
otel-lgtm	3000 (Grafana), 4317, 4318	
lancamentos-service	8081	build a partir de `services/lancamentos-service/Dockerfile`
consolidado-service	8082	build a partir de `services/consolidado-service/Dockerfile`

Dentro da rede do compose, `JWT_ISSUER_URI` deve permanecer `http://localhost:8080/realms/fluxo-caixa` para bater com o `iss` do token obtido pelo host; usar `spring.security.oauth2.resourceserver.jwt.jwk-set-uri` apontando para `http://keycloak:8080/realms/fluxo-caixa/protocol/openid-connect/certs` (variável `JWT_JWK_SET_URI`) e validar o issuer separadamente.

12. AWS (alvo)

> **Revisão 2 (2026-11-10):** região passou para `us-east-1`, borda passou a ser CloudFront + WAF com ALB **interno** via VPC Origin, e há três ambientes. Onde esta seção divergir das seções 14–19, valem as seções 14–19.

- VPC em 3 AZs (subnets públicas, privadas de aplicação e privadas de dados), NAT Gateway (1 em dev, 1 por AZ em prod), VPC endpoints (ECR api/dkr, S3, Logs, SQS, SNS, Secrets Manager, STS).
- CloudFront + WAF (escopo CLOUDFRONT) → VPC Origin → ALB **interno** (ver seção 14). Roteamento por caminho: `/api/v1/lancamentos*` → lancamentos, `/api/v1/consolidados*` → consolidado.
- ECS Fargate: cluster único, um serviço por microsserviço, mínimo 2 tasks em AZs distintas, auto scaling (CPU e `ALBRequestCountPerTarget`), deployment circuit breaker com rollback, Container Insights.
- RDS PostgreSQL **separado por serviço** em prod (isolamento de falha), Multi-AZ, criptografado com KMS, senha gerenciada pelo Secrets Manager.
- ElastiCache Redis/Valkey com réplica e failover automático (consolidado).
- SNS + SQS + DLQ criptografados (KMS) com políticas restritas.
- Cognito User Pool + Resource Server `fluxo-caixa` (escopos acima) + Pre Token Generation Lambda (v2) para incluir `comerciante_id` no access token.
- Observabilidade: CloudWatch Logs, ADOT collector como sidecar (traces → X-Ray, métricas → CloudWatch/AMP), dashboards e alarmes (5xx, latência p99, profundidade da fila, idade da mensagem mais antiga, mensagens na DLQ) com notificação via SNS.
- ECR com scan on push e imagens imutáveis.

13. Metas de NFR (SLOs)

Métrica	Meta
Disponibilidade Lançamentos	99,9% mensal, independente do Consolidado
Disponibilidade Consolidado	99,5% mensal
Latência POST lançamento	p95 < 200 ms, p99 < 500 ms
Latência GET consolidado	p95 < 100 ms (cache), p99 < 300 ms
Throughput Consolidado	≥ 50 req/s sustentados, erro ≤ 5% (meta interna ≤ 1%)
Atraso de consolidação (eventual consistency)	p95 < 5 s entre registro e reflexo no saldo
RPO / RTO	RPO ≤ 5 min (PITR RDS), RTO ≤ 1 h

14. Borda: CloudFront + WAF + VPC Origin (Revisão 2)

- **CloudFront** é a única entrada pública. Distribuição com `PriceClass_All` (necessário para usar os edges da América do Sul, onde estão os comerciantes), HTTP/2 e HTTP/3, TLS 1.2+ (`TLSv1.2_2021`), certificado do ACM em `us-east-1`, domínio customizado parametrizável.
- **AWS WAF** com escopo `CLOUDFRONT` (exige recursos em `us-east-1`): AWSManagedRulesCommonRuleSet, KnownBadInputs, SQLiRuleSet, AmazonIpReputationList, AnonymousIpList (count em dev) e rate-based rule por IP. Logs do WAF em CloudWatch Logs/S3.
- **Origem**: CloudFront **VPC Origin** apontando para **ALB interno** (subnets privadas). O ALB não tem IP público; seu security group aceita apenas o prefix list gerenciado `com.amazonaws.global.cloudfront.origin-facing`.

- **Cache e rotas**: só `/api/v1/lancamentos*` e `/api/v1/consolidados*` vão ao ALB, com `CachingDisabled` + origin request policy `AllViewerExceptHostHeader` (respostas são por comerciante e autenticadas — não cachear na borda). O comportamento default do CloudFront devolve 404 numa função de viewer-request e não chama a origem. O listener do ALB repete a mesma fronteira: as duas regras encaminham, e a ação padrão é 404 fixo. `/actuator` e o Swagger não são rota pública. O health check do target group fala com a task direto, sem passar pelo listener. Ganho do CloudFront aqui é TLS terminado no edge de São Paulo (GRU), conexões persistentes pela backbone AWS até `us-east-1`, absorção de DDoS (Shield Standard), WAF na borda e ponto único para futuros front-ends estáticos (S3 + OAC).
- **HTTP até o ALB**: o TLS da internet termina no CloudFront. O trecho CloudFront → VPC Origin → ALB interno é HTTP na rede da AWS. O hostname do ALB interno não cabe num certificado ACM público, e o security group só aceita o prefix list `com.amazonaws.global.cloudfront.origin-facing`. Não há API em claro na internet.
- Headers de segurança via Response Headers Policy (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy).

15. Ambientes e dimensionamento (us-east-1)

Item	dev	hml	prod
AZs	2	2	3
NAT Gateway	1	1	1 por AZ (3)
VPC interface endpoints	não (sai pelo NAT)	sim, 2 AZs	sim, 3 AZs
lancamentos (Fargate ARM64)	0,5 vCPU / 1 GB, min 1 / max 2, Fargate Spot	0,5 vCPU / 1 GB, min 1 / max 3	0,5 vCPU / 1 GB, min 2 / max 10, on-demand
consolidado (Fargate ARM64)	0,5 vCPU / 1 GB, min 1 / max 2, Fargate Spot	1 vCPU / 2 GB, min 1 / max 3	1 vCPU / 2 GB, min 2 / max 10, on-demand
RDS PostgreSQL 17 (1 por serviço)	`db.t4g.micro`, single-AZ, 20 GB gp3	`db.t4g.small`, single-AZ, 20 GB gp3	`db.t4g.medium`, Multi-AZ, 50 GB gp3. Read replica clássica fica em 0: a AWS não cria réplica quando a senha é gerenciada pelo Secrets Manager
RDS Proxy	não	não	parametrizável (padrão desligado)
ElastiCache Valkey	`cache.t4g.micro`, 1 nó	`cache.t4g.small`, 1 nó	`cache.t4g.small`, primário + réplica, Multi-AZ
CloudFront + WAF	sim	sim	sim
Retenção de logs	7 dias	14 dias	30 dias
Container Insights	desligado	ligado	ligado (enhanced)
Deletion protection / backups	não / 1 dia	não / 7 dias	sim / 14 dias + PITR

Valores são **defaults** nos `*.tfvars` de cada ambiente; tudo é parametrizável.

16. Rastreabilidade ponta a ponta (traces nos logs)

- Propagação **W3C Trace Context** (`traceparent` , `tracestate`) em HTTP (entrada e saída) via Micrometer Tracing + OpenTelemetry bridge.
- Header `X-Correlation-Id` : aceito na requisição (UUID; se ausente/inválido é gerado), devolvido na resposta, colocado no MDC como `correlationId` .
- **Logs JSON** (structured logging do Boot) sempre com: `traceId` , `spanId` , `correlationId` e, quando aplicável, `comercianteId` , `lancamentoId` , `eventId` .
- **Outbox**: no momento do POST, o `traceparent` e o `correlationId` correntes são gravados na linha de `outbox_eventos` (colunas `trace_parent` , `trace_state` , `correlation_id`). O relay, que roda em outra thread, publica no SNS reconstituindo esse contexto (span `PRODUCER` filho do span do POST), para que o trace não se quebre na fronteira assíncrona.
- **Message attributes SNS** (repassados ao SQS pela raw delivery): `eventType` , `traceparent` , `tracestate` (se houver), `correlationId` — todos `String` . O payload JSON do evento (seção 6) não muda.
- **Consumidor SQS**: `ReceiveMessage` com `MessageAttributeNames=All` e `MessageSystemAttributeNames=All` ; extrai o contexto e abre span `CONSUMER` filho do span do produtor; popula MDC (`traceId` , `spanId` , `correlationId` , `eventId` , `comercianteId`) e limpa ao final. Na AWS o X-Ray recebe o trace único POST → relay → SNS → SQS → consolidação.

17. Rotação de segredos

- **RDS**: senha gerenciada pelo Secrets Manager (`manage_master_user_password`) com **rotação automática** configurável (padrão 30 dias em prod/hml, 90 em dev) e chave KMS própria.
- **Contrato de `DB_URL`** : a task recebe sempre `jdbc:postgresql://<host>:5432/<db>` . `DB_READER_URL` , quando existir, usa o mesmo esquema. O serviço aceita também `jdbc:aws-wrapper:postgresql://` e normaliza para um único prefixo: com `DB_SECRET_ARN` no perfil `aws` , a URL efetiva é `jdbc:aws-wrapper:postgresql://...` (prefixo repetido é removido, não duplicado); sem segredo, o prefixo do wrapper é retirado e o driver PostgreSQL usa `DB_USERNAME` / `DB_PASSWORD` .
- **Plugins**: instância Multi-AZ usa `failover,awsSecretsManager` . Se o host for endpoint de RDS Proxy (o nome contém `.proxy-`), o plugin `failover` fica de fora e só `awsSecretsManager` permanece. O failover do wrapper descobre a instância e conecta nela, furando o proxy e o security group da task. O proxy já faz o failover.
- **Read replica clássica**: a AWS não cria read replica de RDS for PostgreSQL quando `manage_master_user_password` está ligado. O módulo recusa `read_replica_count > 0` nessa combinação. O caminho de código `DB_READER_URL` continua pronto. No gatilho de leitura, a saída compatível com senha gerenciada é o RDS Multi-AZ DB Cluster (reader endpoint) ou o Aurora PostgreSQL; a terceira saída é senha fora do Secrets Manager, rotacionada por Lambda. Ver seção 19 e o ADR 0030.
- Hikari `max-lifetime` < intervalo de rotação. A task role recebe `secretsmanager:GetSecretValue` apenas no ARN do próprio segredo + `kms:Decrypt` .

- **Demais**: KMS com rotação anual automática; certificados ACM renovados automaticamente; chaves de assinatura JWT rotacionadas pelo Cognito (serviços buscam JWKS dinamicamente); CI/CD sem chaves de longa duração (GitHub OIDC → IAM role); EventBridge notifica falhas de rotação (`RotationFailed`) no tópico de alertas.

18. Resiliência em código (Resilience4j)

- Versões: Java 25, Spring Boot 4.0.9, Resilience4j 2.4.0 (`resilience4j-spring-boot4` + `spring-boot-starter-aop` / `aspectj` conforme exigido pelo Boot 4.0). Compatibilidade documentada em `documentos-brutos/` e nos READMEs dos serviços, com matriz de versões e gatilho de upgrade para Boot 4.1.
- **lancamentos**: CircuitBreaker + Retry (backoff exponencial com jitter) + TimeLimiter de **1 s** na publicação SNS do relay; quando o circuito abre, o relay adia o lote (eventos permanecem no outbox, nada se perde) — o POST nunca depende do SNS.
- **consolidado**: CircuitBreaker + TimeLimiter de **300 ms** no Redis, com fallback para o banco; Bulkhead de **8** chamadas no consumo SQS; Retry com backoff nas falhas transitórias de banco no consumidor; RateLimiter das consultas **desligado** por padrão.
- **Cliente HTTP do AWS SDK**: os dois serviços usam `url-connection-client` e excluem `apache5-client` . O AWS SDK 2.57.12 traz `httpcore5` 5.3.6 (CVE-2026-54399 e CVE-2026-54428). Tirar o cliente Apache elimina a biblioteca, em vez de piná-la. O long polling do SQS segura o socket por 20 s; o socket timeout do consolidado é 25 s, e o timeout da tentativa da API fica em 32 s. O SNS do lançamentos não faz long poll: socket e tentativa ficam em 2 s, acima do time limiter de 1 s.
- Métricas do Resilience4j expostas no Micrometer/Prometheus; estados do circuito aparecem em `/actuator/health` (sem derrubar readiness).

19. Desempenho de banco e consultas

- **lancamentos**: índice composto `(comerciante_id, data_lancamento, criado_em DESC)` cobrindo a listagem paginada; paginação keyset opcional; particionamento declarativo por mês em `lancamentos` como evolução documentada; `outbox_eventos` com índice parcial `WHERE publicado_em IS NULL` e limpeza periódica (retenção) dos publicados.
- **Leitura/escrita separadas**: variável opcional **`DB_READER_URL`** no esquema `jdbc:postgresql://` ; quando presente, transações `@Transactional(readOnly = true)` são roteadas para o reader (`LazyConnectionDataSourceProxy` + routing data source); ausente, tudo vai ao primário. Read replica clássica de PostgreSQL não convive com senha gerenciada pelo Secrets Manager. O reader compatível, quando o gatilho disparar, é o endpoint de leitura do Multi-AZ DB Cluster ou de um Aurora. Planos medidos: lancamentos e consolidado.
- **consolidado**: PK `(comerciante_id, data)` atende consulta por dia e por período (range scan); saldo acumulado via soma até a data — documentar e, se o volume crescer, materializar `saldo_acumulado` incremental ou snapshots mensais.
- Hikari dimensionado por task, `pg_stat_statements` e Performance Insights habilitados, `statement_timeout` por usuário, `EXPLAIN ANALYZE` das consultas críticas documentado.

20. API Gateway (Revisão 3)

A composição Terraform implementa este desenho quando `enable_api_gateway = true` (padrão em dev, hml e prod). `false` volta ao caminho da seção 14: CloudFront → VPC Origin → ALB interno.

```
Cliente → CloudFront (+ WAF global) → API Gateway REST API regional (+ WAF regional)
        → VPC Link v2 → ALB interno → ECS Fargate
```

O provider AWS 6.72 aceita a integração privada da REST API com VPC Link v2 direto no ALB (`aws_apigatewayv2_vpc_link` + `integration_target` em `aws_api_gateway_integration`). Não há NLB. O VPC Link v1 (`aws_api_gateway_vpc_link`) só aponta para NLB e não foi usado.

A integração privada inclui o nome do stage no path enviado ao backend (`/v1` + `/api/v1/...`). O listener do ALB, com o flag ligado, reescreve `^/v1(/.*)$` para o path do serviço. Sem o prefixo, o rewrite não casa e o path da seção 14 segue inteiro. O hop API Gateway → ALB é HTTP na VPC: o hostname interno do ALB não cabe num certificado ACM público. Timeout de integração: 10 s.

- **Tipo**: REST API **regional** (não edge-optimized — a borda já é o CloudFront). Escolhido sobre HTTP API por suportar **usage plans + API keys por consumidor**, **WAF associado**, **request validation**, **mTLS em domínio customizado**, **canary de stage** e cache opcional. Trade-off de custo (REST ~USD 3,50/milhão vs HTTP ~USD 1,00/milhão) documentado no ADR 0031.
- **Autorização na borda**: Cognito User Pool authorizer validando o JWT e os escopos por rota (`lancamentos:write`, `lancamentos:read`, `consolidado:read`, com o prefixo do resource server). Os serviços **continuam** validando o JWT (defesa em profundidade, zero trust) e extraindo `comerciante_id`.
- **Rotas expostas**: somente `/api/v1/lancamentos`, `/api/v1/lancamentos/{id}`, `/api/v1/lancamentos/{id}/estornos` e `/api/v1/consolidados/diario`, `/api/v1/consolidados/diario/{data}`, com os métodos da seção 7. Nada de `/actuator`, `/v3/api-docs` ou `/swagger-ui` passa pelo gateway.
- **Proteção contra bypass do CloudFront**: CloudFront envia header `X-Origin-Verify` com valor guardado no Secrets Manager (com **rotação**: Lambda grava o novo valor, atualiza o origin custom header do CloudFront e a regra do WAF regional aceitando atual + anterior durante a janela). O WAF regional do API Gateway bloqueia requisições sem o header correto. A origem usa `AllViewerExceptHostHeader` para o `Host` ser o do `execute-api`.
- **Throttling**: limites por stage e por método (ex.: POST lançamentos e GET consolidado com burst/rate próprios, dimensionados acima de 50 req/s de pico em hml e prod; menor em dev) e **usage plans** por consumidor de integração (API key + quota), além do rate-based rule do WAF de borda.
- **Integração privada**: VPC Link v2 para o ALB interno. Timeout de integração 10 s. Path params e os headers `Authorization`, `traceparent`, `tracestate`, `X-Correlation-Id` e `Idempotency-Key` seguem para o serviço. `X-Origin-Verify` é removido nesse hop.
- **Observabilidade**: access logs JSON no CloudWatch (com `requestId`, `extendedRequestId`, `xrayTraceId`, `ip`, `status`, `latency`, `integrationLatency`, `apiKeyId`, além de `traceparent` e `correlationId`), X-Ray tracing ativo no stage, métricas e alarmes (5XX, 4XX, Latency p99, IntegrationLatency).
- **Alternativa parametrizável**: `enable_api_gateway = false` mantém CloudFront → VPC Origin → ALB interno (seção 14).